

Inserting Clock and Reset Synchronization Logic

Introduction

This lab demonstrates how to prevent glitches and metastability in digital systems by inserting reset synchronizer and clock domain crossing (CDC) synchronizer modules. These techniques are essential for building robust hardware where asynchronous signals (such as resets or inter-domain communications) must be reliably brought into a local clock domain. You will learn to integrate synchronizers, simulate, and visualize their effectiveness.

Objective

To demonstrate how inserting reset synchronizer and CDC synchronizer modules prevents glitches and metastability during asynchronous signal transfer and reset deassertion.

Procedure to create a lab:

Create a lab directory `sync_logic_demo`

Navigate to `sync_logic_demo` directory and create the files for `reset_sync`, `cdc_sync`, `top_module_with_sync`, `testbench`.

Step-by-Step lab creation:

Lab 1: 2-Stage Reset Synchronizer

The reset synchronizer ensures that the asynchronous reset signal (`async_reset`) deasserts in alignment with the local clock domain (`clk`), avoiding glitches in downstream logic.

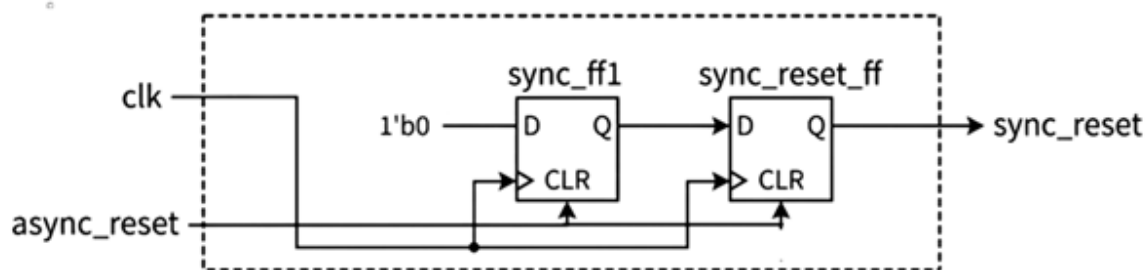


Fig: 2-stage reset synchronizer

Create a file for reset synchronizer: `reset_sync.v`

// Converts an asynchronous reset into a safe, glitch-free synchronous reset

```
module reset_sync (
    input clk,                // Destination clock domain
    input async_reset,        // Asynchronous reset input
    output reg sync_reset ); // Synchronized reset output
```

```
reg sync_ff1;                                // First flip-flop for capturing async_reset

// Use both posedge clk and posedge async_reset for immediate assertion

always @(posedge clk or posedge async_reset) begin

    if (async_reset) begin

        // Immediate assertion of reset in both flops

sync_ff1      <= 1'b1;

        sync_reset    <= 1'b1;

    end else begin

        // Deassertion happens synchronously across clock edges

        sync_ff1      <= 1'b0;

        sync_reset    <= sync_ff1;

    end

end

endmodule
```

Lab 2: 2-Stage Data Synchronizer (CDC Logic)

A CDC synchronizer (clock domain crossing) ensures reliable capture of asynchronous signals (async_in) into the local clock domain (clk), mitigating the risk of metastability by using two flip-flops.

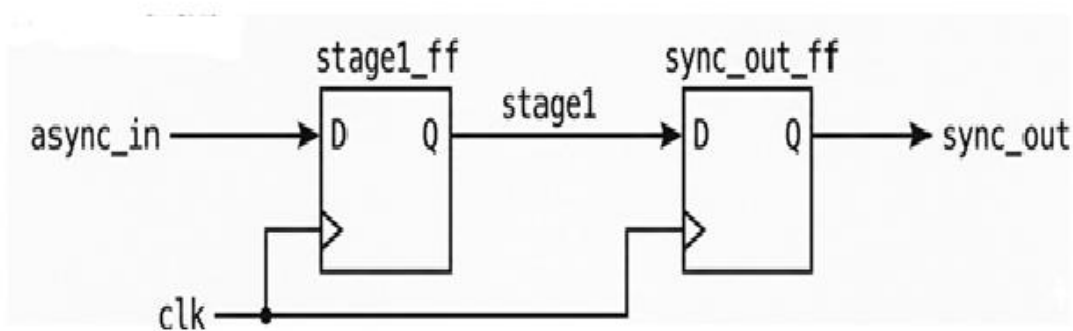


Fig: 2-Stage Data Synchronizer

Create a file named `cdc_sync.v`

// Safely captures an asynchronous data input into the clk domain

```
module cdc_sync (  
    input clk,                // Destination clock  
    input async_in,          // Asynchronous input signal  
    output reg sync_out      // Synchronized output  
);  
  
    reg stage1;              // Intermediate register  
  
    // 2-stage flip-flop synchronizer  
    always @(posedge clk) begin  
        stage1  <= async_in;    // First flop captures async signal  
        sync_out <= stage1;     // Second flop settles metastability  
    end  
endmodule
```

Lab 3: Integration of Reset and Signal Synchronizers

This top module instantiates both synchronizers, simulating a common SoC scenario where resets and CDC signals must be cleaned before use.

Create a file named `top_with_sync.v`

// Top module that integrates reset and CDC synchronizers

```
module top_with_sync (  
    input clk,                // Main clock input  
    input async_reset,        // Global asynchronous reset  
    input async_signal,       // CDC input from another domain  
    output synced_reset,      // Cleaned reset  
    output synced_signal      // Cleaned signal  
);  
  
    // Instantiate the reset synchronizer  
    reset_sync u_reset_sync (  
        .clk(clk),
```

```
.async_reset(async_reset),  
.sync_reset(synced_reset)  
);  
  
// Instantiate the 2-stage CDC synchronizer  
cdc_sync u_cdc_sync (  
    .clk(clk),  
    .async_in(async_signal),  
    .sync_out(synced_signal)  
);  
endmodule
```

Step 4: Testbench to Verify Synchronization Logic

async_signal is pulsed asynchronously (not aligned with the clock), and async_reset changes at non-clock-aligned times. The outputs allow you to observe how the synchronizers safely bring these signals into the clock domain.

Create the testbench sync_tb.v

// Testbench to verify reset and CDC synchronization behavior

```
module sync_tb;  
  
    reg clk = 0;           // System clock  
    reg async_reset = 1;   // Start with reset active  
    reg async_signal = 0;  // Default async input  
    wire synced_reset;  
    wire synced_signal;  
  
    // Generate clock  
    always #5 clk = ~clk;  
  
    // Instantiate DUT  
    top_with_sync dut (  
        .clk(clk),
```

```
.async_reset(async_reset),  
.async_signal(async_signal),  
.synced_reset(synced_reset),  
.synced_signal(synced_signal)  
);  
  
initial begin  
    $dumpfile("dump.vcd");  
    $dumpvars(0, sync_tb);  
  
    // Deassert reset after some time  
    #12 async_reset = 0;  
  
    // Inject asynchronous signal pulses  
    #15 async_signal = 1;  
    #3  async_signal = 0;  
  
    #20 async_signal = 1;  
    #2  async_signal = 0;  
  
    #40 $finish;  
end  
endmodule
```

Create Simulation script

a) Create "run.sh"

```
#!/bin/bash
```

Step 1: Run Verilator to compile and generate simulation files

```
verilator --binary -j 0 -Wall \  
    reset_sync.v cdc_sync.v top_with_sync.v sync_tb.v \  
    --top sync_tb --timing --CFLAGS "-std=c++20" --trace
```

Step 2: Navigate to the object directory

```
cd obj_dir || { echo "obj_dir not found"; exit 1; }
```

Step 3: Compile the simulation binary

```
make -f Vsync_tb.mk Vsync_tb || { echo "Compilation failed"; exit 1; }
```

Step 4: Run the simulation

```
./Vsync_tb || { echo "Simulation failed"; exit 1; }
```

Step 5: Open the waveform in GTKWave

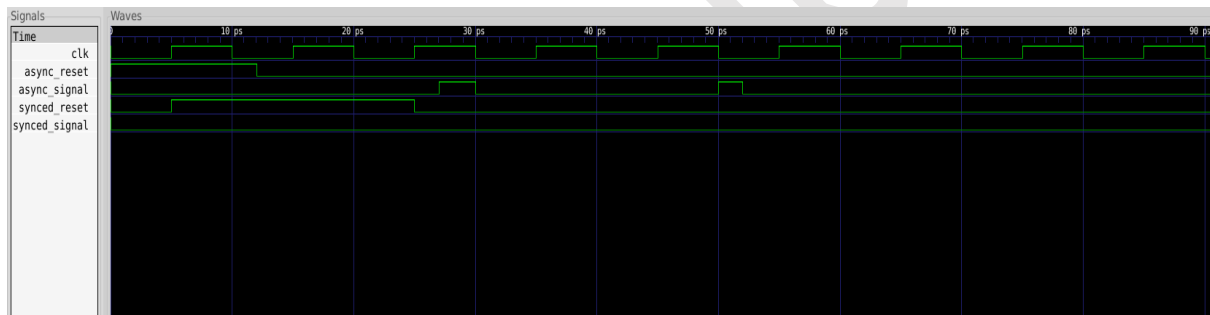
```
gtkwave dump.vcd
```

b) Make the script executable:

```
chmod +x run.sh
```

c) Run the script (./run.sh) to perform compilation, simulation, and open the waveform.

GTKWave Waveform:



Signal	Observation
async_reset	Deasserted abruptly, may not align to clock edge
syncd_reset	Deasserts cleanly, aligned to clk
async_signal	Pulses at arbitrary times, may be very short/glitchy
syncd_signal	Clean, clock-aligned assertion a cycle or two later

- async_reset might toggle at any time, but syncd_reset will always release only on a clock edge.
- async_signal pulses happen at irregular intervals, but syncd_signal detects them cleanly, free of glitches or metastability.

Key Learnings of the lab:

Concept	Description
Asynchronous Reset	May cause glitches or meta-states when deasserted unsynchronized
Reset Synchronizer	Ensures async resets are safely released in the clock domain
Clock Domain Crossing	Unsafe transitions can cause metastability
CDC Synchronizer	Two flip-flops in the destination clock absorb meta risks
RTL Integration	Modular composition using synchronizer IP blocks
GTKWave Debugging	Visualization of correct/incorrect signal transitions
Verilator Simulation	Simulation and verification of the synchronization system

Conclusion

This lab session illustrated the challenges and solutions for handling asynchronous resets and inter-domain signals in digital systems. By implementing and integrating 2-stage synchronizers for both reset and data signals, you observed how these modules prevent glitches and metastability, ensuring reliable operation during clock domain crossings or abrupt reset deassertions. Visualization in GTKWave reinforces how the synchronizer output always conforms to the local clock's timing, regardless of when the asynchronous input toggles.